What do professional software developers need to know to succeed in an age of Artificial Intelligence?

Matthew Kam
Google
Mountain View, CA, USA
mattkam@google.com

Cody Miller
Google
San Francisco, CA, USA
codymi@google.com

Miaoxin Wang
Trilyon
Seattle, WA, USA
mxworking@gmail.com

Abey Tidwell
Google
San Francisco, CA, USA
atidwell@google.com

Irene A. Lee
Education Development Center
Waltham, MA, USA
ilee@edc.org

Joyce Malyn-Smith
Education Development Center
Waltham, MA, USA
jmsmith@edc.org

Beatriz Perret
Boston College
Chestnut Hill, MA, USA
beatriz.perretg@gmail.com

Vikram Tiwari
Assembled
San Francisco, CA, USA
hi@vikramtiwari.com

Joshua Kenitzer
Google
Buffalo, NY, USA
jkenitzer@google.com

Andrew Macvean
Google
Kirkland, WA, USA
amacvean@google.com

Erin Barrar
Google
Cambridge, MA, USA
erinbarrar@google.com

Abstract

Generative AI is showing early evidence of productivity gains for software developers, but concerns persist regarding workforce disruption and deskilling. We describe our research with 21 developers at the cutting edge of using AI, summarizing 12 of their work goals we uncovered, together with 75 associated tasks and the skills & knowledge for each, illustrating how developers use AI at work. From all of these, we distilled our findings in the form of 5 insights. We found that the skills & knowledge to be a successful AI-enhanced developer are organized into four domains (using Generative AI effectively, core software engineering, adjacent engineering, and adjacent non-engineering) deployed at critical junctures throughout a 6-step task workflow. In order to "future proof" developers for this age of AI, on-the-job learning initiatives and computer science degree programs will need to target both "soft" skills and the technical skills & knowledge in all four domains to reskill, upskill and safeguard against deskilling.

CCS Concepts

• **Software and its engineering** → **Software development techniques**; • **Computing methodologies** → **Artificial intelligence**; • **Social and professional topics** → **Computing education**.

FSE Companion '25, Trondheim, Norway

ACM ISBN 979-8-4007-1276-0/2025/06
https://doi.org/10.1145/3696630.3727251

Keywords

DACUM, Generative Artificial Intelligence, Human-Centered AI, Software Engineering Education

ACM Reference Format:
Matthew Kam, Cody Miller, Miaoxin Wang, Abey Tidwell, Irene A. Lee, Joyce Malyn-Smith, Beatriz Perret, Vikram Tiwari, Joshua Kenitzer, Andrew Macvean, and Erin Barrar. 2025. What do professional software developers need to know to succeed in an age of Artificial Intelligence?. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), June 23–28, 2025, Trondheim, Norway*. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3696630.3727251

1 Introduction

Generative Artificial Intelligence (GenAI) is disrupting both the profession and practice of software engineering. The popular press [8, 24, 26, 35] is replete with questions about the career relevance of learning to code when Large Language Models (LLMs) are demonstrating capabilities such as code generation. As of 2024, AI-powered developer tools such as GitHub Copilot are widely used by over 1.3 million paid users [34]. Randomized controlled trials of AI-powered developer tools in both lab [45] and real-world, industry work settings [9] show AI's promising impact on developer productivity. For instance, professional developers who use GitHub Copilot to carry out everyday, enterprise-level work for durations lasting between two and seven months exhibit a 26% productivity improvement as measured by the number of Pull Requests completed weekly [9]. In other words, for those developers whose responsibilities involve only coding, once augmented with AI, the same volume of work that used to take a business week can be performed in a 4-day workweek. More importantly, given AI's likelihood to generate code that needs to be reviewed and edited, increased coding speed

is not attained at the expense of a detectable impact on code quality [9].

More troubling, however, the productivity gains from AI do not appear to be distributed equally among developers. A sparse body of research on the skills & knowledge to be a successful developer in the age of GenAI [2, 4] suggests that developers need prerequisite skills and knowledge in order to leverage AI tools productively. For instance, the most junior developers – with under one year of experience – take 7-10% longer to complete their tasks in some situations when using AI tools compared to not using AI [4]. The same study recommends that this segment of developers receive "additional coursework in foundational programming principles - for example, coding syntax, data structures, algorithms, design patterns, and debugging skills" [4] in order to use AI developer tools productively. Similarly, developers who are more skilled respectively at software engineering – including requirements engineering – are more successful at using Large Language Models (LLMs) for building production quality systems [43] and requirement elicitation tasks [2].

Specifically, our research questions are:

- **RQ1.** Among developers who are experts at using GenAI to accelerate their work, what are the tasks that they use GenAI for at work?
- **RQ2.** What skills & knowledge are essential for developers to effectively leverage GenAI for their tasks at work?

This paper contributes to our understanding of the skills & knowledge to be a successful developer when leveraging AI-powered developer tools. In the rest of this paper, we first survey industry sources and academic research on what is currently known about GenAI in software development. We then detail our research process for developing an "occupational profile" [51] of the AI-enhanced developer. This process (DACUM, short for "Developing A CurriculUM") has been widely used to create occupational profiles that are subsequently used for job analysis, workforce training, and the development of national occupational skill standards. Next, we report on 12 work-related goals that developers on the cutting edge of using AI-powered developer tools are using AI to accelerate in their jobs, together with 75 tasks that we discover they carry out with AI to achieve these 12 goals. We share 5 insights on how AI is reshaping the work of a developer, and the skills & knowledge needed to thrive in AI-enhanced software development. We show how these skills & knowledge are deployed in tandem along a 6-step workflow that applies to any of the 75 tasks. Finally, we discuss the implications for both higher education and on-the-job learning to future-proof developers for the age of GenAI.

As an international expert group across academia, industry and government notes: "A human-centered approach to AI that … interacts with individuals while *respecting human's cognitive capacities* [our emphasis]" is one of the six grand challenges in human-centered AI [15]. In order to adopt a human-centered AI perspective on software engineering and software engineering education, we take a nuanced approach toward the developer user's cognitive capacities by situating the skills & knowledge for the age of GenAI within the following three core principles:

- **Principle 1. Facilitate Reskilling:** Enable developers to adapt their skills & knowledge to new and changing tools, including AI-powered ones, in order to *maintain* their productivity.
- **Principle 2. Enable Upskilling:** Provide opportunities for developers to deepen their expertise and advance mastery of their craft, thereby *increasing* their productivity and promoting their career growth.
- **Principle 3. Safeguard Against Deskilling:** Prevent automation from displacing developers in the short-term at tasks associated with opportunities to develop the skills & knowledge essential for long-term growth and productivity [3]. This risk associated with "skill loss" and undermining junior workers from growing into a pipeline of expert workers applies to all industries impacted by AI, and is estimated to be a "multi-trillion-dollar problem" [3].

Our research contribution is significant in at least three ways. Firstly, with 63% of professional developers already using AI [49], the challenge in inventorying the use cases for AI tools – together with the skills & knowledge for success with these tools – necessitates a qualitative "deep dive" with a small sample of professional developers who are *experts* at AI-enhanced software development. Such experts are likely to be found among early adopters of AI tools. Below, we describe both our stringent criteria and process lasting five months for recruiting such expert informants. Our empirical research with a panel of experts adds to the validity of what individual expert accounts [33, 43] already report on the skills & knowledge crucial for AI-enhanced development. Secondly, existing curricula sources – such as those reviewed in Related Work below – are more likely to introduce the use of AI tools for writing code and tests, or selected stages in the software development lifecycle that are more suitable for teaching someone to become a software engineer. In contrast, this paper reports on 75 tasks – and their corresponding skills & knowledge – that span the *entire* lifecycle encompassing the conceptualization, development, deployment and maintenance of production systems. Thirdly, and along the same line, existing curricula sources mostly target pedagogical objectives without considerations about legacy code, for instance. In contrast, professional developers often work with existing, mature codebases. The skills & knowledge reported in this paper are comprehensive in accounting for what it takes to build software under a range of real-world, industry conditions.

2 Related Work

Recent studies have aimed to understand how and why professional developers use AI tools. A Stack Overflow survey found 63% of professional developers around the world currently use AI tools and, of those that use AI tools, the most common uses are to write code (82%), look for answers (68%), debug (57%), produce documentation (40%), generate assets & "synthetic data" (35%), understand code (31%) and test code (27%) [49]. Similarly, a survey with a convenient sample of 73 experts in Agile development methodologies uncovered a total of 17, 11 and 2 tasks respectively performed using AI that can be classified under hands-on technical work, engineering project management and team management [37]. Next, use cases that are more novel include using AI to arrive at system requirements [2], manage software development projects [7, 53], make

context-aware changes to pasted code [17], migrate variable types in large codebases [38], and repair broken builds [23].

The reported benefits of using AI tools include higher task completion rates [12] and faster completion times for tasks – as much as 45-50% when documenting code and 20-30% when refactoring code [4] – and increased productivity as measured by weekly Pull Requests [9]. Importantly, increased coding speed was not attained at the expense of a detectable impact on code quality [4, 9]. Another reported benefit of AI tool use is that developers can shift their time to work on other, less repetitive tasks such as designing systems [28], meeting customer needs [4, 28], collaborating with team members [28] and learning [28]. Other benefits are seen in broader measures of developer productivity [16]: developers who use AI tools experience increased satisfaction [4, 12, 18], greater time spent on work that is more fulfilling [4, 12] and longer time in a state of "flow" [4, 12, 18] while expending less cognitive effort [12]. Beside productivity gains, accelerating the pace of learning is one of the most frequently cited benefits of AI tools, reported by 61% of professional developers [49].

Conversely, recent research also uncovered developers' insights on the limitations and potential harms of using AI for software development. Stack Overflow found that only a minority of professional developers concur that AI tools will increase code accuracy (29%), make workloads more manageable (24%), and improve collaboration (8%) [49]. They also found that fewer than 12% of developers fear job loss due to AI which aligns with other findings that 31% of developers question the accuracy of their existing AI tools, and nearly 45% of them believe their existing AI tools to be unsuitable for complex tasks [49]. Other frequently cited limitations of AI are its lack of understanding of context about the codebase, the organizational context, and inability to decompose a complex problem into smaller parts [4]. Furthermore, as a result of using GitHub Copilot, developers are spending less time on Stack Overflow, and have a decreased understanding about how and why the code works [5].

Given that AI's benefits and limitations depend on the capabilities of the actual AI tools, some research has been conducted to compare AI tools that are applicable to those stages of the software development lifecycle prior to system deployment (e.g. requirements, design, implementation) [27].

Currently, there is little research on the linkages between specific skills & knowledge and the effective use of GenAI in software development. McKinsey's research [4] finds that the most junior developers – those with less than one year of experience – take 7-10% more time to complete their work in some situations when they are using AI tools. The authors of the McKinsey study conclude that Generative AI tools are "only as good as the skills of the engineers using them" and "for developers to effectively use the technology to augment their daily work, they will likely need a combination of training and coaching" [4]. This finding is consistent with the prior observation that the developers who are more skilled at requirements gathering and validation are the same developers more likely to use LLMs effectively to elicit requirements [2]. Next, while these accounts do not constitute formal empirical research, there are nonetheless individual expert accounts of the skills & knowledge that are deemed crucial [25, 33, 43].

Prompt engineering features prominently in the skills & knowledge highlighted in existing work. For instance, prior work has contributed [39, 55] and evaluated [47] prompt patterns that developers can use to effectively leverage LLMs for tasks in the phases of the software development lifecycle prior to deployment (e.g. requirements elicitation, system design, implementation, refactoring). Where hands-on knowledge is concerned, recent Massive Open Online Courses and hands-on programming books on AI-enhanced software development introduce developers to LLM prompts for specific software development tasks [1, 14, 36, 39, 46, 50] across some phases in the development lifecycle [36, 46], selected GenAI tools for these tasks [1, 14, 39, 46, 50] and the fundamentals of machine learning (including GenAI) [13, 36, 50].

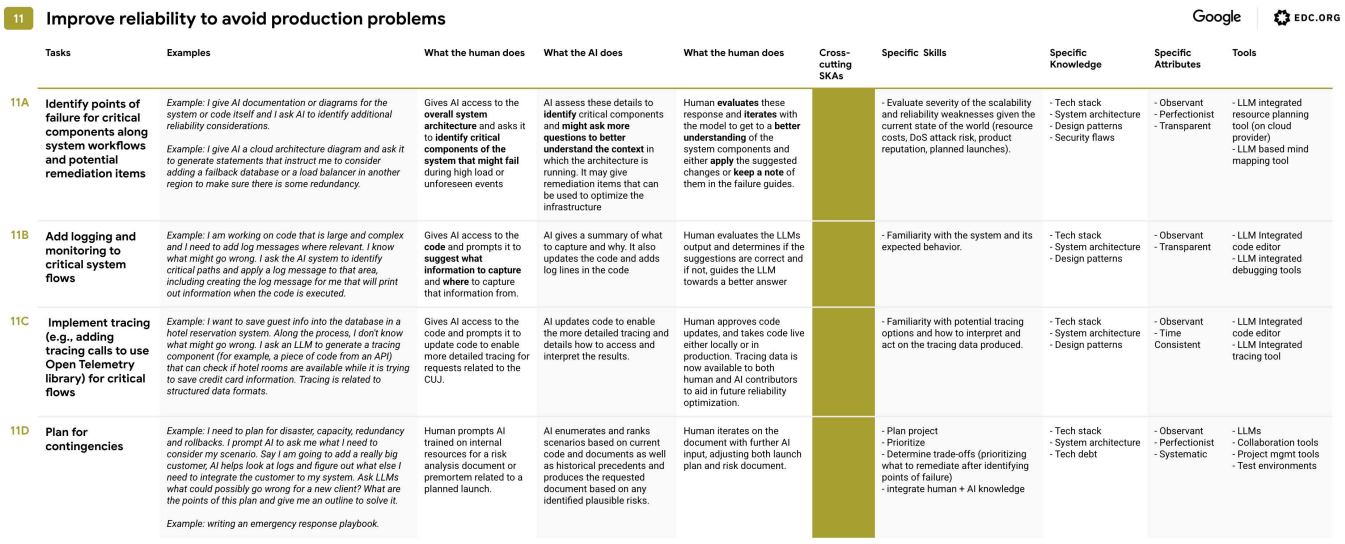
Yet, effective prompt engineering also entails understanding the attributes that make up quality code, and knowing how to prompt the AI tool for the right outputs to "actively iterate" on AI-generated code until it meets the desired quality [4, 10]. McKinsey's research concludes that developer training "should equip developers with an overview of generative AI risks, including any industry-specific data privacy or intellectual-property issues and best practices in reviewing AI-assisted code for design, functionality, complexity, coding standards, and quality, including how to discern good versus bad recommendations from the [AI] tools" [4]. That is, the effective use of AI in software development goes beyond knowing how to use LLMs.

3 Developing the Occupational Profile: Research Design

We chose to use the "Developing A CurriculUM" (DACUM) methodology [40, 42] that originated in the late 1960s [22] because it is a widely accepted method for job analysis, workforce training, and the development of national occupational skill standards. DACUM has been applied across more than 120 occupations in over 58 countries for more than 40 years [51]. The DACUM process rests upon three basic principles: 1) Expert workers can describe and define their jobs more accurately than anyone else; 2) An effective way to define a job is to precisely describe the tasks that expert workers perform; and 3) All tasks, in order to be performed correctly, demand certain knowledge, skills, resources, and behaviors. Thus a key first task is to identify and recruit a panel of 6-10 expert informants, which is the ideal number of informants required to run a DACUM workshop [52].

3.1 Expert Informants Recruitment

Our greatest challenge in defining a new or emerging occupation such as AI-enhanced software development was to identify and recruit those early adopter individuals where experts in the emerging field are more likely to be found. For this effort we sought expert informants who 1) were experienced developers on the cutting-edge of using AI tools to accelerate productivity at work; 2) came from workplaces where AI coding tools have been rolled out in teams across the organization, since collaboration is a significant contributor to complexity in software engineering [6]; 3) had extensive experience using GenAI coding tools day-to-day for at least 6 months; 4) submitted Pull Requests (PRs) several times per week



11 Improve reliability to avoid production problems

Google | EDC.ORG

	Tasks	Examples	What the human does	What the AI does	What the human does	Cross-cutting SKAs	Specific Skills	Specific Knowledge	Specific Attributes	Tools
11A	**Identify points of failure for critical components along system workflows and potential remediation items**	*Example: I give AI documentation or diagrams for the system or code itself and I ask AI to identify additional reliability considerations.* *Example: I give AI a cloud architecture diagram and ask it to generate statements that instruct me to consider adding a failback database or a load balancer in another region to make sure there is some redundancy.*	Gives AI access to the **overall system architecture** and asks it to **identify critical components of the system that might fail** during high load or unforeseen events	AI assess these details to **identify** critical components and **might ask more questions to better understand the context** in which the architecture is running. It may give remediation items that can be used to optimize the infrastructure	Human **evaluates** these response and **iterates** with the model to get to a **better understanding** of the system components and either **apply** the suggested changes or **keep a note** of them in the failure guides.		- Evaluate severity of the scalability and reliability weaknesses given the current state of the world (resource costs, DoS attack risk, product reputation, planned launches).	- Tech stack - System architecture - Design patterns - Security flaws	- Observant - Perfectionist - Transparent	- LLM integrated resource planning tool (on cloud provider) - LLM based mind mapping tool
11B	**Add logging and monitoring to critical system flows**	*Example: I am working on code that is large and complex and I need to add log messages where relevant. I know what might go wrong. I ask the AI system to identify critical paths and apply a log message to that area, including creating the log message for me that will print out information when the code is executed.*	Gives AI access to the **code** and prompts it to **suggest what information to capture** and **where** to capture that information from.	AI gives a summary of what to capture and why. It also updates the code and adds log lines in the code	Human evaluates the LLMs output and determines if the suggestions are correct and if not, guides the LLM towards a better answer		- Familiarity with the system and its expected behavior.	- Tech stack - System architecture - Design patterns	- Observant - Transparent	- LLM Integrated code editor - LLM integrated debugging tools
11C	**Implement tracing (e.g., adding tracing calls to use Open Telemetry library) for critical flows**	*Example: I want to save guest info into the database in a hotel reservation system. Along the process, I don't know what might go wrong. I ask an LLM to generate a tracing component (for example, a piece of code from an API) that can check if hotel rooms are available while it is trying to save credit card information. Tracing is related to structured data formats.*	Gives AI access to the code and prompts it to update code to enable more detailed tracing for requests related to the CUJ.	AI updates code to enable the more detailed tracing and details how to access and interpret the results.	Human approves code updates, and takes code live either locally or in production. Tracing data is now available to both human and AI contributors to aid in future reliability optimization.		- Familiarity with potential tracing options and how to interpret and act on the tracing data produced.	- Tech stack - System architecture - Design patterns	- Observant - Time Consistent	- LLM Integrated code editor - LLM Integrated tracing tool
11D	**Plan for contingencies**	*Example: I need to plan for disaster, capacity, redundancy and rollbacks. I prompt AI to ask me what I need to consider my scenario. Say I am going to add a really big customer, AI helps look at logs and figure out what else I need to integrate the customer to my system. Ask LLMs what could possibly go wrong for a new client? What are the points of this plan and give me an outline to solve it.* *Example: writing an emergency response playbook.*	Human prompts AI trained on internal resources for a risk analysis document or premortem related to a planned launch.	AI enumerates and ranks scenarios based on current code and documents as well as historical precedents and produces the requested document based on any identified plausible risks.	Human iterates on the document with further AI input, adjusting both launch plan and risk document.		- Plan project - Prioritize - Determine trade-offs (prioritizing what to remediate after identifying points of failure) - integrate human + AI knowledge	- Tech stack - System architecture - Tech debt	- Observant - Perfectionist - Systematic	- LLMs - Collaboration tools - Project mgmt tools - Test environments

Figure 1: This Figure shows the portion of the occupational profile of the AI-enhanced software developer that corresponds to four tasks (e.g. identify points of failure for critical components) for achieving the goal "improve reliability to avoid production problems." More broadly, the occupational profile developed using the DACUM process uses a grid to show each of the 13 goals that developers use AI to perform at work, and the tasks that are performed to meet each goal. Each task is illustrated using example tasks as well as examples of what the developer does to interact with AI, what the AI does in response, and what the developer does in turn in response to the AI. These examples were gathered from both the panelists and advisors. For each of the tasks, the profile also presents a list of the specific skills, knowledge, attributes & tools necessary to succeed at the given task. Some of the skills, knowledge & attributes (SKAs) are cross-cutting and apply to all tasks. The cross-cutting SKAs are not detailed in this Figure and will be described more in Sections 4.2 and 4.3.

that involved AI-generated code, and submitted PRs that have undergone significant prior iterations since the original AI-generated code; and 5) reported that AI has a "major" or "transformative" impact on their software development workflows, could share at least one use of AI coding tools that is novel (e.g. not reported in the literature), and contributed to their colleagues' understanding and use of AI tools through on-the-job mentoring or other technical leadership activities.

We set out to build a diverse panel of experts who represented various genders, industries, and company sizes. Informant recruitment painstakingly occurred over five months concurrently over three distinct channels and was non-blind. For our first channel, we enlisted C2 Research, an agency which specializes in recruiting research participants. C2 winnowed over 3,000 developers across industry down to 300 who expressed interest, had no conflicts of interest, and were available on the tentative workshop dates. We further winnowed down to 16 who appeared to meet the above five recruiting criteria, who were then phone screened by the 5th author, a researcher with a background in software development and 20 years of experience in Computer Science Education. 7 expert AI-enhanced developers were recruited from this channel and participated in the workshop. Our second channel, the Google Developer Experts (GDE) community [20] of more than 1,000 developers and technologists employed outside of Google who are thought leaders, influencers and experts on Google technologies yielded one additional US-based panelist. This panel of 8 experts from outside Google who were recruited through the first and second channels participated in the principal phase of the DACUM process (i.e. phase 2), which is the workshop where a first draft of the occupational profile of the AI-enhanced software developer was produced. (More details about all 3 DACUM phases are provided in the next subsection.)

Though extensive efforts were made for all channels to reach women who were developers experienced at using AI tools, we were unable to find enough of them who met all the five criteria above. In particular, of the three women who were phone screened, only one met all five criteria. Thus, while we sought greater gender diversity, our final panel comprised 7 males and 1 female. We believe this gender imbalance on the panel reflects a broader industry imbalance in gender diversity at the forefront of AI-use by developers. Recent research found that women comprise only 22% of AI talent globally, with representation at senior levels at less than 14% [44]. The gender disparity is extreme in the AI industry: women comprise only 15% of AI research staff at Facebook and 10% at Google [48]. The industries represented on the panel were ecommerce, cybersecurity, entertainment, food services, and scientific computing. In terms of company size, 2 panelists were from companies with between 1-5 employees; 1 panelist from a company with between 11-50 employees; 1 panelist from a company with between 200-500 employees; 1 panelist from a company with between 1000-3000 employees; and 2 panelists were from companies with over 10,000 employees.

Our third channel – in-house subject matter experts within Google – was used to identify and recruit 13 subject matter experts (all males) within Google who participated as advisors in both phase 1 (familiarization) and phase 3 (validation). We received 200 applications (176 males, 24 females), and winnowed down to 16 applicants who met most of the above screening criteria. In particular, we prioritized those who had at least 6 months of early adopter experience with AI coding tools outside of Google (in addition to those AI coding tools being designed and deployed within Google). Through our rigorous recruitment process with these three channels using the above criteria, we are confident that our 8 participating panelists and 13 advisors are *expert-level* AI-enhanced developers who could effectively describe a comprehensive set of work tasks in AI-enhanced software development and the skills, knowledge & dispositions needed to perform that work effectively.

3.2 Three-Phased Profile Development Process

DACUM involves three major phases: 1) exploring the target occupation; 2) producing an occupational profile [22, 41]; and 3) validating the profile.

Phase 1: Exploring prior to DACUM workshop. Phase 1's goal was to arm ourselves with as much understanding as possible — at a concrete level -– about AI-enhanced software development, before we set out to conduct the DACUM workshop (Phase 2). We drafted a straw definition of an AI-enhanced software developer drawn from prior research. We asked our 13 advisors to identify how they were already using AI to enhance business value in Google's 30 high-level software development goals. They provided rich details including screenshots showing exactly how they used AI to accelerate those goals, reflected on how AI has changed their work, and whether AI has been a net positive or negative change for their productivity.

Phase 2: DACUM workshop. Adapting the DACUM process which we used successfully in the past to develop profiles for emerging fields [11, 21, 29–32, 54], Phase 2 was a 2.5-day workshop held Friday - Sunday (May 31 - June 2, 2024), so that panelists do not have to take "paid time off" to participate in the research. The 8 panelists first reviewed and edited the straw occupational definition, then voted to identify 11 of the Google developer goals and 2 new goals they added to best describe their work as AI-enabled software developers. When vetted, these goals were found to be highly consistent with the same developer goals from Phase 1 that our advisors believed AI to be best positioned to enhance business value. Next, panelists broke down the 13 goals into smaller tasks and provided examples of what each task looks like when enabled with AI and then worked in pairs to list the knowledge, skills, attributes & tools necessary corresponding to each task. They presented their work for group consensus. The workshop ended with a focus group on how AI has impacted software development thus far and future trends. At the conclusion of the workshop, we had a draft occupational profile (figure 1) which included the occupational definition; a grid of 13 goals and their corresponding 91 tasks with examples; and for each of the 91 tasks, a list of the knowledge, skills, attributes, and tools necessary to succeed at the task. At the end of the workshop, panelists each agreed that the profile effectively captured their work as AI-enhanced developers.

Phase 3: Qualitatively validating the profile after DACUM workshop. Phase 3's goal was to ensure that the DACUM-styled profile, aimed at educators "Developing A CurriculUM", would also contain sufficient detail and be understood as the panelists intended by a technical audience that included professional software developers and computer science professors. The profile resulting from Phase 2 had some ambiguities because the 30 developer goals presented to panelists during the workshop contained ambiguous language which trickled down into the tasks, and because some language in the profile was meant to allow for flexibility in interpretation. As a result, we needed to support one uniform interpretation of all 13 goals by re-organizing some tasks and editing some of the technical language. For every statement in the profile that had multiple possible interpretations, we converged on the best interpretation with the advisors and 8th author (who was one of the original panelists and had context from earlier workshop discussions). Then we carefully and painstakingly edited the profile to include "signpost" language on the relevant "engineering parameter" (e.g. granularity, metrics, lifecycle stage, developer goal) where the panelists presumably intended flexibility of interpretation, so that the reader can more confidently exercise flexibility when interpreting the profile. In total, we proposed to reduce the total number of distinct tasks from 91 to 80 once there was consensus that 11 particular tasks contained sufficient duplication with other tasks. All panelists had the opportunity to "sign off" on the proposed edits before the profile was finalized on October 17, 2024.

4 Research Findings

4.1 Addressing RQ1: Among developers who are experts at using GenAI to accelerate their work, what are the tasks that they use GenAI for at work?

Among the 13 goals discovered during the DACUM workshop, in our professional opinion, the 13th goal (i.e. testing AI models) represents a highly specialized activity. Developers who need to *build* AI-powered software features – vis-a-vis *use* AI/LLM-based features to accelerate their work – are much more likely to undertake this goal. That is, success for this goal hinges on having the specialized skills & knowledge of an AI or Machine Learning (ML) engineer. Since this paper focuses on the broader practice of software development – whether or not the developer is an AI/ML specialist – we will not discuss this 13th goal in this paper (although this goal is found in the above occupational profile). A total of 75 tasks in the profile correspond to the 12 goals that we will discuss in the rest of this paper.

Specifically, the following 12 goals and pertinent[1] tasks in the profile apply widely to the practice of software development:

(1) **Contextualize a unit of work that needs to be done** comprehensively by leveraging AI to summarize voluminous user and stakeholder feedback, investigate competitors' offerings for strengths and gaps, and acquire more context about how the system behaviors they need to implement

[1] In the interest of space, we will not enumerate all of the 75 tasks that are performed to meet all 12 goals. Instead, we focus on describing those tasks that even an audience who's familiar with AI-enhanced software development are more likely to find novel.

fit into the overall system by prompting AI to explain the existing codebase and the locations in the overall codebase that relate to their work (e.g., file, lines of code) based on the context available to AI (8 AI-enhanced developer tasks in total for this goal)

(2) **Locate information related to this unit of work** (e.g. documentation, open source libraries, codelabs, API examples) by using AI to research the latest technologies for meeting the necessary technical requirements to deliver a compelling value proposition; locate and learn about the necessary technical knowledge, possible tools, and libraries for implementation (2 tasks)

(3) **Explore technical solutions for approaching this work** by leveraging AI to optimize for the given technical metrics (e.g. performance, complexity, cost, maintenance, reliability) and evaluate various technical options based on their advantages and downsides (8 tasks)

(4) **Develop and document a considered plan for completing this work** by using AI to help draft a high-level implementation roadmap, including the visuals, dependencies and other references (8 tasks)

(5) **Produce high-quality code** by using AI to remove complexity in the codebase and to convert design documentation – including visual schematics of how different components interact and how the user-interface ought to look – from idea to code (8 tasks)

(6) **Create and maintain holistic test coverage** by asking AI to evaluate test coverage, identify points of failure, recommend test cases, and generate test data that reflect real-world or stress-testing conditions (7 tasks)

(7) **Generate assets** (e.g., captions, images) by leveraging AI tools to research terminologies (e.g., styles, references) and optimize prompts iteratively based on various input and output modalities (e.g., reference image, text) (3 tasks)

(8) **Produce up-to-date documentation** by asking AI to generate accompanying inline code comments; assess necessary documentation changes; and generate the updated documentation, including user-facing examples (e.g., API tutorial for various use cases that a developer may need to implement) (4 tasks)

(9) **Ensure launch complies with legal, privacy and security** by leveraging AI to assess potential privacy violations, patent and/or copyright infringements, generate penetration tests, simulate those tests in order to identify anomalies, and propose possible solutions (7 tasks)

(10) **Monitor data and generate insights** for important events (e.g., changes in response time, error metrics post deployment, production interaction logs) by using a combination of GenAI and more traditional approaches (e.g. rule-based thresholds, statistical outlier detection, time-series analysis) to identify anomalies and generate predictions on the outcome of taking a given approach (9 tasks)

(11) **Investigate issues in production** by providing AI with code, system and error logs, stack traces and even user-interface outputs to conceptually determine the sources of the issues and isolate the responsible code and/or configurations (7 tasks)

(12) **Improve reliability to avoid production issues** by leveraging AI to analyze the codebase, proactively identify likely failure conditions, and recommend appropriate logging strategies, short-term mitigation measures, and long-term risk mitigation plans (4 tasks).

The above 12 goals and corresponding 75 tasks are carried out using AI tools that include LLMs embedded as features in popular developer tools such as GitHub Copilot, conversational LLMs like ChatGPT, multimodal LLMs, and other developer tools based on pre-GPT Machine Learning models.

Thinking about where the above 12 goals and 75 tasks are situated along the software development lifecycle provides us with an additional perspective for understanding how GenAI is changing the practice of software development. Specifically, in the lifecycle, Goals 1 to 4 primarily revolve around learning about the problem and design space, investigating technical options, and making decisions about these options prior to implementation. Goals 5 to 8 center around the production of high quality code, documentation, and other AI-generated artifacts. Goals 9 to 12 are about ensuring a compliant, robust production, launch and post-launch pipeline. To make all of the above clearer, we briefly summarize the key tasks that an expert-level AI-enhanced developer uses AI to carry out in each stage of the lifecycle, as further illustrated in Figure 2:

- **Plan (58 AI-enhanced developer tasks in total for this stage)**: engaging in the process of project-level [2], technical-level planning[3], and investigation[4] prior to detailed implementation.
- **Code (42 tasks)**: engaging in the process of generating code, developing tests, and configuring the necessary system setup.
- **Build (11 tasks)**: engaging in the process of transforming source code into a deployable artifact.
- **Test (28 tasks)**: engaging in the process of evaluating the system at various levels and using multiple types of testing to ensure quality.
- **Release (8 tasks)**: engaging in the process of making a version of the system available.
- **Deploy (7 tasks)**: engaging in the process of making a system operational in a specific environment.
- **Operate (27 tasks)**: engaging in the ongoing process of keeping a system running smoothly. It overlaps with other stages when tasks involve coordination and task management work. One example task is triaging active production issues which is also mapped to both the "Operate" and "Plan" stages.

[2]Project planning in this paper refers to the overall management and execution planning of the project. The process of defining the scope, objectives, and resources required, estimating effort and resources, and risk management.

[3]By technical planning, we mean the process of defining the technical blueprint of the system. To have a detailed design plan, it requires parallel work in "solution investigation" together to explore implementation options, comparing them, and selecting the optimal solution based on various constraints and trade-offs. Comparing to "solution investigation", this stage focuses on the "what" and "why"

[4]The solution investigation process bridges the gap between the "what" (the high-level tech and project plan) and the "how" (the actual code). It's where the technical details are fleshed out and the feasibility of different approaches is thoroughly examined. This stage emphasizes practical experimentation, prototyping, and risk assessment.

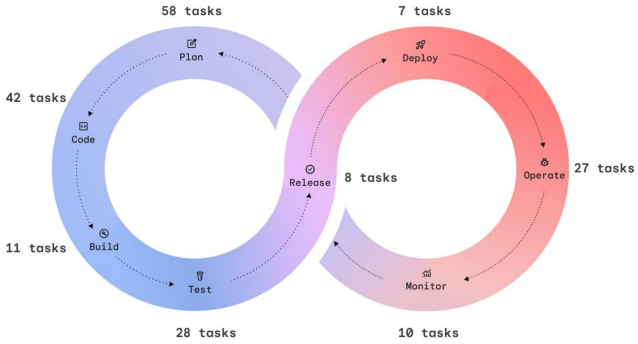


Figure 2: One way to better understand and appreciate how AI is changing the practice of software development is to consider which of the 75 AI-enabled developer tasks apply to each stage of the software development lifecycle. The number of tasks across all stages total to more than 75 because the tasks are not mutually exclusive. That is, several tasks are applicable to more than one stage. One insight that this lifecycle-based visualization offers is that as much as the literature reports on how developers are benefiting from GenAI for coding and testing, developers on the cutting edge of using AI are carrying out even more tasks in the "plan" stage with the help of AI.

- **Monitor (10 tasks)**: engaging in the ongoing process of observing and tracking metrics (e.g., availability, error, performance, user behaviors).

With less time spent on repetitive work, AI frees developers to experiment – under shorter cycles – and come up with optimal solutions to problems. From the profile, we observe that developers seek to use AI to optimize for both business metrics (e.g. "selects one system that is most vetted, most efficient, cost effective and has better chance of success and profitability") and engineering metrics (e.g. "complexity, performance, maintenance") **(insight #1)**. In other words, the future of software development augmented using AI is likely to entail an increased emphasis on meeting the intersection of business and technical goals.

> **One workshop panelist**: *To me it's … like we have a specific problem and we think we can get there and then [we think] … of something else and - oh, I would love to do that. And now we have the time [to explore an alternate solution] because AI has reduced the amount of time spent on what we normally would do.*
> **Moderator**: *So it seems like you're saying rather than just find a solution, find the best solution.*
> **Panelist**: *Find the best because [yes] it's like [we find] a solution, but maybe there's a better one.*

In some situations, AI tools facilitate individual developers in meeting their team's greater good. For example, generating example code for tutorial documentation (task) when producing up-to-date documentation (goal) can be time-consuming in the short term. With AI, developers complete this task more efficiently, and potentially reduce disruptions and save even more time in the long term – for themselves and coworkers – when coworkers can find answers from better documentation instead of having to ask **(insight #2)**.

Finally, while early research indicates that AI's time savings promote collaboration [28], our research suggests that the reality is more complex **(insight #3)**. When AI helps with finding previous code authors and explaining code, there is a reduced need for direct communication. Panelists report spending less time communicating with others. Similarly, an advisor notes that unless AI is used with care (e.g. AI-generated summaries of emails), human interactions can be reduced to information exchanges devoid of the human touch. On the other hand, when AI saves time asking and answering simpler questions, AI creates the potential for human interactions to focus on more meaningful questions. The same advisor adds that AI can help to put people in touch: *"AI more like a 'trusted advisor' rather than an 'executive' would probably be more of a net positive: less 'Here is a summary of your coworkers doc' and more 'Hey @ldap, we think this doc written by @foo relates to your existing work and plans.'"*

4.2 Addressing RQ2: What skills & knowledge are essential for developers to effectively leverage GenAI for their tasks at work?

In this subsection, we present a bigger picture of what these skills & knowledge look like when aggregated across all 12 goals and 75 tasks. The full dataset of skills & knowledge from the occupational profile by goal or task can be found at arXiv.org. In summary, the skills & knowledge to be an effective developer in the age of GenAI can be organized under four distinct domains, as illustrated using a T-shape diagram in Figure 3:

Domain 1. Generative AI Usage: This domain revolves around effective engagement with AI tools, including being able to assess AI's outputs and calibrating interactions with LLMs to achieve desired outcomes (see section 4.3 for more details). This domain also includes foundational knowledge of machine learning; as well as AI tools' capability and constraints (e.g., AI failures, including how and why AI hallucinates), which enable developers to select appropriate tools for given tasks and avoid wasting time using AI due to unrealistic expectations about AI.

Domain 2. Core Software Engineering: This encompasses the essential skills and knowledge for software development, which remain crucial for making sound technical decisions in the age of GenAI. Three key aspects about this domain include 1) coding and testing proficiency – knowing what constitutes high-quality maintainable code, best practices in defensive coding, optimization, systematic testing approaches, and debugging skills with comprehension and critical thinking; 2) risk assessment for production readiness (e.g., being able to recognize potential technical compromises, vulnerabilities, and planning for contingencies); and 3) good system design (e.g., possessing a deep understanding of existing and alternative system architectures, design & system constraints, and carefully weighing the potential benefits and drawbacks of multiple design solutions to meet requirements).

Domain 3. Adjacent Software Engineering: Specialized subdomains within and closely related to software engineering e.g. cybersecurity, industry-specific regulations, and emerging technological trends. The exact topics depend on the developer's context.

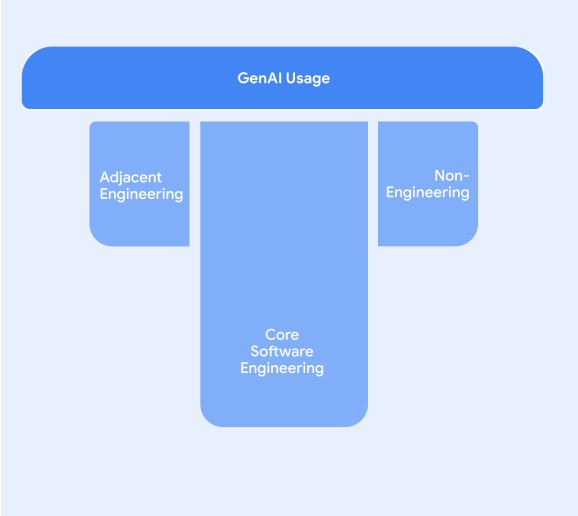


Figure 3: The T-shape is a common visualization used to represent the skills & knowledge to be an expert worker in a field. The vertical bar of the "T" represents the deep expertise in a particular area that's core to success in the professional role. The horizontal bar in the "T" represents a wider breadth of knowledge & skills that complement the core, deep area of expertise. Knowing how to use GenAI effectively (dark blue) is the first knowledge & skills domain for the AI-enhanced developer to use AI productively at work, which in turn hinges on three additional domains (light blue). Specifically, the expertise of the AI-enhanced developer is characterized by depth in "core software engineering". This deep expertise is applied most effectively at work when complemented by breadth in adjacent domains that are both related and non-related to engineering. In practice, the exact skills & knowledge that goes into each of the four domains, and the relative importance – and size – of each domain, depends on organizational factors including how job roles are defined by a particular employer.

For instance, in organizational settings that require software engineers to develop areas of specialization, a front-end developer focused on experimentation with proof-of-concept prototypes may not require knowledge of scale parameters for assessing and optimizing system resources for deployment planning.

Domain 4. Adjacent Non-Software Engineering: This encompasses at least 5 distinct sub-domains as we learn from the profile: end-user, customer, business or industry, competitors landscape, and market trends. Similar to Domain 3, the specific topics (e.g. General Data Protection Regulation) depend on the organizational context. For example, developers in smaller companies may need the skills & knowledge to "translate product (and technical) requirements into business needs (and vice versa)", "analyze feedback from users and internal stakeholders" and "integrate needs and feedback into work item". Developers in larger companies may focus more on contextualizing their work items so they are better able to evaluate trade-offs (e.g., cost-benefit analysis) and risks when optimizing for business value.

With the rise of AI, developers are changing how they acquire content knowledge **(insight #4)**. As one advisor puts it: *"Before GenAI... I'd have to maybe **read books** [respondent's emphasis]... While I [still] need to be familiar with other [programming] languages, [this] can be an issue for junior developers because I acquired this [knowledge]... by ... reading."*

4.3 A new workflow common across all tasks that weaves together the skills & knowledge

Regarding the skills & knowledge that are cross-cutting across all 75 tasks, we first observe that every task in the profile is carried out along a 6-step task workflow when the developer is collaborating with AI. In describing each of the 6 steps below, we use a color code to underscore where the corresponding skills & knowledge domains [5] essential for successfully completing the step are deployed in the workflow:

(1) **Identify**: Developers start an interaction by seeking and identifying the information that AI tools need (**Domain 1**), based on their knowledge of AI tools' capability (**Domain 1**) and the context of the task – from engineering (**Domain 2**, **Domain 3**) to business (**Domain 4**).
(2) **Engage**: They clearly express (**1**) a need (e.g., write a structured request that yields the specific information or product you are seeking) with the corresponding context (**2**, **3**, **4**), problem statement (**2**, **3**, **4**), and desired outcomes (**2**, **3**, **4**).
(3) **Evaluate**: They critically cross-check AI-generated artifacts, that may include content, designs, code, templates, or diagrams, for AI failures (**1**) against their domain knowledge (**2**, **3**, **4**). They verify for discrepancies against their expected end results, and other criteria for success (**2**, **3**, **4**).
When applicable, they put these artifacts in action (e.g., sandbox) to observe and compare. They note the limitations of the AI (**1**) and areas to improve its output (**2**, **3**, **4**).
(4) **Calibrate**: They steer AI tools toward the desired end results (**2**, **3**, **4**) with feedback (**1**) and additional context such as the the system (**2**), product (**4**), goals (**2**, **3**, **4**), and potential tradeoffs (**2**, **3**, **4**).
(5) **Tweak**: They take AI-generated artifacts and improve them based on their knowledge of the expected standards to meet (**2**, **3**, **4**).
(6) **Finalize**: Having arrived at the final version of the desired artifact, they produce up-to-date documentation (when applicable) with clarity and accuracy, such as reporting on how and why they arrived at the final version of the artifacts (**2**, **3**, **4**).

In practice, the steps in the above workflow may be *optional*, *iterative* and/or *non-linear*. Depending on the actual situation at work, when a developer carries out one of the 75 tasks with the help of AI, some of the above steps may be skipped (i.e. optional). Similarly, a developer may repeat a given step multiple times until the desired output is obtained from the AI (i.e. iterative). Finally,

[5]The underlying skills & knowledge supporting interactions with AI are marked using the following color code: **Domain 1 Generative AI**, **Domain 2 Core software engineering**, **Domain 3 Adjacent engineering**, **Domain 4 Adjacent non-engineering**

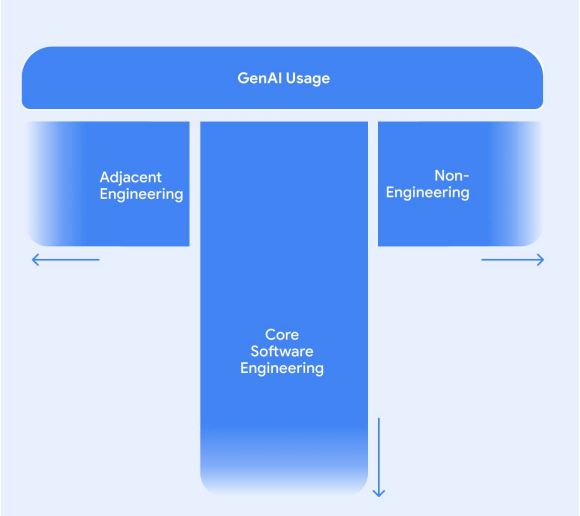


Figure 4: The AI-enhanced developer (of tomorrow) will need to have the skills & knowledge of today's senior developer. From an upskilling perspective, developers will need to deepen their depth of skills & knowledge in the "core software engineering" domain, as well as widen their breadth of skills & knowledge in adjacent domains both related and non-related to engineering, in order to fully realize the benefits of AI tools while managing the risks.

in order to be flexible based on what occurs during a given step, a developer may not perform the above workflow linearly from step 1 to step 6, and may need to "backtrack" to a previous step until the conditions are in place to advance to the next step (i.e. non-linear).

The use of GenAI tools is perhaps often understood to entail prompting an LLM only. Unfortunately, this corresponds to only a subset of the Engage step in the above 6-step task workflow.[6] The significance of the above task workflow is that it highlights the existence of other steps (e.g. Calibrate, Tweak) in an end-to-end workflow that are often essential for the successful completion of the overall, AI-enhanced task. Most importantly, the above description of the workflow shows where in the workflow, and how, Domains 2, 3 & 4 are absolutely critical for developers to effectively leverage AI tools. This perspective goes beyond prompt engineering in presenting a holistic understanding of the skills & knowledge that are critical for AI-enhanced software development. In fact, it calls for fluency in these skills & knowledge in order for the developer as a user of AI to be effective as the human in the loop, exercising appropriate control over AI throughout the workflow **(insight #5)**.

5 Implications

The media raises the specter of the human developer being displaced by AI. As reported in Related Work above, however, we are beginning to see the potentially transformative benefits of AI. As such, in our view, the perspective to take is less around the risk of AI displacing the developer, as much as the risk of the AI-enhanced developer displacing the developer who does not *reskill* to keep up with AI tools and be capable of realizing the maximum productivity benefits they offer. There is an utmost urgency for professional software developers to be equipped with the skills and knowledge to succeed in the age of GenAI, such that the human developer is capable of being in the loop at all times, ensuring that the benefits of AI are realized while its risks are managed. This starts with on-the-job learning opportunities as well as computer science degree programs going to the heart of where and how **GenAI usage (i.e. Domain 1)** appears throughout the above 6-step task workflow, reskilling developers for a changing toolchain and task workflow in which AI tools play an increasingly prominent role.

Next, when developers use AI to complete repetitive work with less time or effort, the time savings enable them to spend more time in the "plan" stage of the software development lifecycle (see Figure 2). As we have learned, AI-augmented tasks in the latter stage include understanding business and user requirements; translating these requirements into the corresponding technical requirements, metrics, and technical decisions; exploring alternative solutions (e.g. system design, architecture) for achieving better results on key metrics; and weighing possible solutions to arrive at the most suitable one. In this way, the AI-enhanced developer is delegating work that is more repetitive to AI, while spending more time in the role of a technical decision maker who reviews suggestions from AI.

Today, senior developers assume the entire range of the above responsibilities that include system design and making sound technical decisions, whereas junior developers are not ready to take on the fullest extent of these responsibilities until they *upskill* and acquire the depth of competency in **"core software engineering" (i.e. Domain 2)**. In other words, for a developer to fully realize the productivity benefits of AI, a regular AI-enhanced developer of tomorrow will need to have the skills & knowledge of today's senior developer (see Figure 4). There is an impetus for on-the-job learning opportunities as well as computer science degree programs to provide a deeper foundation in the skills & knowledge essential for good system design and technical decision making.

As we have also learned, developers receive support from AI to further their agency in going beyond solving engineering problems to designing and implementing the best solutions to business and user problems. In the process, developers cross domain boundaries. Using General Data Protection Regulation (GDPR) as an example of an adjacent, non-engineering domain; developers cross boundaries from core software engineering to an adjacent domain to build an implementation that is legally compliant. Specifically, they use AI to arrive at a basic understanding of the technical requirements, before they gain access to colleagues with expertise in the legal domain (e.g. in a big company setting) or in the absence of colleagues with legal expertise (e.g. in a small company setting).

The implication is that developers must *upskill* by also broadening their skills & knowledge in **adjacent domains related to engineering (i.e. Domain 3)** as well as **adjacent domains non-related to engineering (i.e. Domain 4)** (see again Figure 4). While computer science degree programs cannot feasibly teach everything, their breadth requirements can be reviewed to ensure that

[6] We would also note that as AI agents become increasingly popular in a developer's workflow, the human-AI interaction in the Engage step may evolve from how prompt engineering is currently done. Yet, our 6-step task workflow remains relevant in an agentic world.

students have ample opportunities to acquire the necessary breadth of knowledge & skills both within, and more importantly, outside of engineering – aligned with Domain 4's five sub-domains above. This prepares students to meet the needs of the future that we cannot even imagine today.

In the age of GenAI, AI-enhanced developers who have stronger "soft skills" – in addition to the technical skills & knowledge we have just discussed – have a competitive edge over other developers. For example, strong communication skills enable developers to effectively seek help from coworkers once they have determined, based on their expertise with AI tools, that they have reached the limits of what AI can do to help with their tasks. Similarly, while AI can be double-edged sword in its impact on human relationships, on the whole, excellent collaboration skills enable developers to forge more meaningful relationships at work. Importantly, where instructional time is concerned, emphasizing soft skills does not need to come at the expense of technical skills & knowledge. Both on-the-job learning opportunities and computer science degree programs can investigate ways to better prioritize, structure and integrate supports for growing teamwork skills into existing curricula that target technical skills & knowledge.

Finally, AI fuels the impetus for the developer to be a continuous learner, especially of content knowledge. Users of LLMs need to continually evaluate the accuracy, relevance, etc. of the LLM's response. The discourse on AI literacy emphasizes the importance of skills such as critical thinking for recognizing hallucinations or misinformation. What is less often acknowledged or even recognized is that for users to critically evaluate the LLM's response, they need the necessary content knowledge [19] in AI, core software engineering, and adjacent domains for skills such as critical thinking to draw on and be successfully applied. Worse, with developers increasingly learning from AI-generated content instead of reading human sources (e.g. Stack Overflow [5]) whose content is more authoritative, the erosion of skills & knowledge appear to be beginning. To *safeguard against deskilling*, providers of both lifelong learning and higher education will need to consider more ways to increase access to high quality content.

6 Conclusions

Prior to the rise of AI, business-centered problem solving, collaborative teamwork, and iteration & experimentation are longstanding themes in software development. Similarly, developers already draw on skills & knowledge across domains whenever they can. What's new about AI is that technological disruption is amplifying these trends. Within the wider discourse on the automation of knowledge work, our research most strongly supports the view that AI is blurring the boundaries of what a professional developer does, in ways that augment rather than replace the developer. AI empowers the developer to design and build technology solutions that best solve problems for the user, customer and business. To the extent that the organizational context calls for the developer to become more business centric, work will likely transform to be even more collaborative, iterative, and experimentation-driven in pursuit of this aim.

Essential to the developer's success in this future of work is the ability to continually acquire new skills & knowledge —- including content knowledge. This knowledge base is multidimensional in that it spans how to effectively use LLMs at work, core engineering domains, as well as domains in and outside of engineering adjacent to core engineering. The multidimensional nature of this knowledge & skills supports developers in their work, where we see boundaries begin to blur in two ways. Firstly, AI is providing developers with the agency to more directly understand the business and user, and to more effectively incorporate these requirements into system designs at multiple levels of the technology stack. Secondly, developers where possible draw on this multidimensional knowledge base in a way that increasingly blurs the boundaries between their core engineering and adjacent domains.

7 Limitations

This study has four main limitations. The first limitation is the small sample size. The use of AI in software engineering was nascent at the time of the workshop, thus it was difficult to find expert informants who met our criteria for duration and depth of AI use. Of those we found, all were "early adopters" and of limited diversity and thus our findings may not be generalizable to all developers who eventually adopt AI tools for software engineering or as the field evolves. A second limitation is the inability to conduct a statistical validation of the findings. Attempts to do so were hampered by the limited number of potential validators. A third limitation is the dearth of prior research studies on this topic that would anchor our research in the past. Finally, a fourth limitation is that our study relies on self-report of AI use and was not independently verified. To mitigate potential sources of bias (such as selective memory, attribution bias, or exaggeration), we drilled down and asked informants to recount specific examples of AI tool use within each task. Despite these limitations, we argue that there is utility in this research; since software engineering is a field where rapid early adoption of AI is occurring, the findings from this study may offer important implications for on-the-job learning initiatives and degree programs related to software engineering; as well as new, potentially useful information for other fields that are moving forward with AI adoption.

Acknowledgments

The first author expresses his gratitude to his family for their support when working on this paper. This research would not have been possible without the expert contributions of the 8 panelists and 13 advisors. We thank Allison Beck, Martina Blahova, Jacinda Ashley Jones, Shelly Bogetich Klose, Carmen Musat, Jeremy Neuner and CoreyMarie Roberts for their utmost assistance with expert informants recruitment; Megan Dizon, Aparna Kadakia and Laurie Wu for organizational support with the DACUM workshop; Joseph Ippito for help in planning and facilitating the workshop; James Lin and Mauli Pandey for their contributions to the literature search; and Monica Chan for her guidance with Overleaf. Finally, we thank Heather Breslow, Tao Dong, K.C. Geiger, Collin Green, Henning Fisher, Daye Nam, Yan Schober, Darla Sharp and Kevin Storer for constructive feedback on earlier drafts of this work; and Mohamed Dekhil, Maggie Johnson, Brian Saluzzo, Shivani Govil and Sara Ortloff for their executive sponsorship.

References

[1] Rav Ahuja, Antonio Cangiano, and Ramanujam Srinivasan. 2024. *Generative AI for software developers.* Coursera. Retrieved 2025-01-13 from https://www.coursera.org/specializations/generative-ai-for-software-developers

[2] Chetan Arora, John Grundy, and Mohamed Abdelrazek. 2024. Advancing requirements engineering through generative AI: Assessing the role of LLMs. In *Generative AI for Effective Software Development*, A Nguyen-Duc, P Abrahamsson, and F Khomh (Eds.). Springer Nature, Cham, Switzerland, 129–148. https://doi.org/10.1007/978-3-031-55642-5_6

[3] Matthew Beane. 2024. *The Skill Code: How to Save Human Ability in an Age of Intelligent Machines.* (first edition ed.). HarperBusiness, New York, NY.

[4] Begum Karaci Deniz, Chandra Gnanasambandam, Martin Harrysson, Alharith Hussin, and Shivam Srivastava. 2023. *Unleashing developer productivity with Generative AI.* McKinsey Digital. Retrieved 2024-09-20 from https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai

[5] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2023. Taking flight with Copilot. *Commun. ACM* 66 (may 2023), 56–62. Issue 6.

[6] Frederick P Brooks, Jr. 1995. *The mythical man-month* (2 ed.). Addison Wesley, Boston, MA.

[7] Beatriz Cabrero-Daniel, Yasamin Fazelidehkordi, and Ali Nouri. 2024. How can generative AI enhance software management? Is it better done than perfect? In *Generative AI for Effective Software Development*, A Nguyen-Duc, P Abrahamsson, and F Khomh (Eds.). Springer Nature, Cham, Switzerland, 235–255. https://doi.org/10.1007/978-3-031-55642-5_11

[8] Hasan Chowdhury. 2024. *Software engineers are getting closer to finding out if AI really can make them jobless.* Business Insider. Retrieved 2025-01-16 from https://www.businessinsider.com/cognition-labs-devin-ai-software-engineer-humans-lose-jobs-2024-3

[9] Zheyuan Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz. 2024. The effects of Generative AI on high skilled work: Evidence from three field experiments with software developers. Retrieved 2024-09 from https://www.ssrn.com/abstract=4945566

[10] Arghavan Moradi Dakhel, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Hironori Washizaki. 2024. Generative AI for software development: A family of studies on code generation. In *Generative AI for Effective Software Development*, A Nguyen-Duc, P Abrahamsson, and F Khomh (Eds.). Springer Nature, Cham, Switzerland, 151–172. https://doi.org/10.1007/978-3-031-55642-5_7

[11] Education Development Center and Bunker Hill Community College. 2021. *Profile of a security operations analyst level 1.* Education Development Center.

[12] Eirini Kalliamvakou and and Github staff. 2024. *Research: Quantifying GitHub Copilot's impact on developer productivity and happiness.* The GitHub Blog. https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/

[13] Aymen El Amri. 2023. *LLM prompt engineering for developers: The art and science of unlocking LLMs' true potential.* Independently Published, FAUN Community.

[14] Aymen El Amri. 2024. *The augmented developer: Code smarter, not harder.* Independently Published, FAUN Community.

[15] Ozlem Ozmen Garibay et al. 2023. Six Human-Centered Artificial Intelligence Grand Challenges. *International Journal of Human–Computer Interaction* 39, 3 (Feb. 2023), 391–437. https://doi.org/10.1080/10447318.2022.2153320

[16] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of Developer Productivity. *ACM queue: tomorrow's computing today* 19 (feb 2021), 20–48. Issue 1.

[17] Simone Forte and Marcus Revaj. 2024. *Smart Paste for context-aware adjustments to pasted code.* Google Research. Retrieved 2024-10-15 from https://research.google/blog/smart-paste-for-context-aware-adjustments-to-pasted-code/

[18] Ya Gao and GitHub Customer Research. 2024. Research: Quantifying GitHub Copilot's impact in the enterprise with accenture. Retrieved 2024-09-22 from https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-in-the-enterprise-with-accenture/

[19] Roberta Michnick Golinkoff and Kathryn Hirsh-Pasek. 2016. *Becoming brilliant: What Science Tells Us About Raising Successful Children.* American Psychological Association, Washington, D.C., DC.

[20] Google for Developers. 2024. Google Developer Experts. Retrieved 2024-09-25 from https://developers.google.com/community/experts

[21] J Ippolito, M A Latcovich, and J Malyn-Smith. 2008. *In fulfillment of their mission: The duties and tasks of a Roman Catholic priest: An assessment project.* National Catholic Educational Association.

[22] Ronald L Jacobs. 2019. Job Analysis and the DACUM Process. In *Work Analysis in the Knowledge Economy: Documenting What People Do in the Workplace for Human Resource Development.* Springer International Publishing, Cham, 63–79. https://doi.org/10.1007/978-3-319-94448-7_5

[23] Emily Johnston and Stephanie Tang. 2024. *Safely repairing broken builds with ML.* Google Research. Retrieved 2024-10-15 from https://research.google/blog/safely-repairing-broken-builds-with-ml/

[24] Hessie Jones. 2024. *The automation takeover: Are software engineers becoming obsolete?* Forbes. Retrieved 2025-01-16 from https://www.forbes.com/sites/hessiejones/2024/09/21/the-automation-takeover-are-software-engineers-becoming-obsolete/

[25] Eunsuk Kang and Mary Shaw. 2024. tl;dr: Chill, y'all: AI Will Not Devour SE. (sep 2024). arXiv:2409.00764 [cs.SE] http://arxiv.org/abs/2409.00764

[26] Sarah Kessler. 2024. *Should you still learn to code in an A.i. world?* The New York times. Retrieved 2025-01-15 from https://www.nytimes.com/2024/11/24/business/computer-coding-boot-camps.html

[27] Dae-Kyoo Kim. 2024. Comparing proficiency of ChatGPT and bard in software development. In *Generative AI for Effective Software Development*, A Nguyen-Duc, P Abrahamsson, and F Khomh (Eds.). Springer Nature, Cham, Switzerland, 25–51. https://doi.org/10.1007/978-3-031-55642-5_2

[28] Kyle Daigle and GitHub staff. 2024. Survey: The AI Wave Continues to Grow on Software Development Teams. Retrieved 2024-08-29 from https://github.blog/news-insights/research/survey-ai-wave-grows/

[29] J Malyn-Smith and J Ippolito. 2014. *Profile of a Big-Data-Enabled Specialist [White paper].* Oceans of Data Institute.

[30] J Malyn-Smith and J Ippolito. 2017. *Profile of the Data Practitioner [White paper].* Oceans of Data Institute.

[31] Joyce Malyn-Smith and Irene Lee. 2012. Application of the occupational analysis of computational thinking-enabled STEM professionals as a program assessment tool. *The Journal of Computational Science Education* 3 (jun 2012), 2–10. Issue 1.

[32] J Malyn-Smith, I A Lee, and J Ippolito. 2017. *Profile of a CT Integration Specialist.* Siu-cheung KONG The Education University of Hong Kong, Hong Kong.

[33] Martin Fowler. 2023. An Example of LLM Prompting for Programming. Retrieved 2024-08-31 from https://martinfowler.com/articles/2023-chatgpt-xu-hao.html

[34] Microsoft Indonesia News Center. 2024. Driving AI Transformation with GitHub Copilot: DANA and Microsoft's Collaborative Efforts in AI-Enhanced Financial Inclusion – Indonesia News Center. Retrieved 2024-09-06 from https://news.microsoft.com/id-id/2024/04/24/driving-ai-transformation-with-github-copilot-dana-and-microsofts-collaborative-efforts-in-ai-enhanced-financial-inclusion/

[35] Christopher Mims. 2023. *What Will AI Do to Your Job? Take a Look at What It's Already Doing to Coders.* The Wall Street Journal. https://www.wsj.com/articles/ai-jobs-replace-tech-workers-8f3dc92

[36] Laurence Moroney. 2025. *Generative AI for software development.* Coursera. Retrieved 2025-01-13 from https://www.coursera.org/professional-certificates/generative-ai-for-software-development

[37] Anh Nguyen-Duc and Dron Khanna. 2024. Value-based adoption of ChatGPT in agile software development: A survey study of Nordic software experts. In *Generative AI for Effective Software Development*, A Nguyen-Duc, P Abrahamsson, and F Khomh (Eds.). Springer Nature, Cham, Switzerland, 257–273. https://doi.org/10.1007/978-3-031-55642-5_12

[38] Stoyan Nikolov and Siddharth Taneja. 2024. *Accelerating code migrations with AI.* Google Research. Retrieved 2024-10-15 from https://research.google/blog/accelerating-code-migrations-with-ai/

[39] C Noring, A Jain, M Fernandez, A Mutlu, and A Jaokar. 2024. *AI-assisted programming for web and machine learning: Improve your development workflow with ChatGPT and GitHub Copilot.* Packt Publishing.

[40] Robert E. Norton. 1997. *DACUM Handbook.* Number 67 in Leadership Training Series. Center on Education and Training for Employment, The Ohio State University.

[41] Robert E. Norton. 1998. Quality Instruction for the High Performance Workplace: DACUM.

[42] Robert E. Norton and John R. Moser. 2008. *DACUM handbook (3rd ed.).* Center on Education and Training for Employment, The Ohio State University.

[43] Addy Osmani. 2024. *The 70% problem: Hard truths about AI-assisted coding.* Elevate. Retrieved 2025-01-16 from https://addyo.substack.com/p/the-70-problem-hard-truths-about

[44] Siddhi Pal, Ruggero Marino Lazzaroni, and Paula Mendoza. 2024. *AI's missing link: The gender gap in the talent pool.* Interface. Retrieved 2025-01-15 from https://www.stiftung-nv.de/publications/ai-gender-gap

[45] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. 2024. How much does AI impact development speed? An enterprise-based randomized controlled trial. *arXiv [cs.SE]* (oct 2024). arXiv:2410.12944 [cs.SE] http://arxiv.org/abs/2410.12944

[46] S Pereira. 2025. *Generative AI for software development.* O'Reilly Media.

[47] Krishna Ronanki, Beatriz Cabrero-Daniel, Jennifer Horkoff, and Christian Berger. 2024. Requirements engineering using generative AI: Prompts and prompting patterns. In *Generative AI for Effective Software Development*, A Nguyen-Duc, P Abrahamsson, and F Khomh (Eds.). Springer Nature, Cham, Switzerland, 109–127. https://doi.org/10.1007/978-3-031-55642-5_5

[48] Tom Simonite. 2018. *AI is the future—but where are the women?* Wired. Retrieved 2025-01-15 from https://www.wired.com/story/artificial-intelligence-researchers-gender-imbalance/

[49] Stack Overflow. 2024. Stack Overflow Developer Survey: AI. Retrieved 2024-08-13 from https://survey.stackoverflow.co/2024/ai/

[50] T Taulli. 2024. *AI-assisted programming: Better planning, coding, testing, and deployment.* O'Reilly Media.

[51] The Ohio State University. 2024. DACUM International Training Center. Retrieved 2024-09-04 from https://cete.osu.edu/programs/dacum-international-training-center/

[52] The Ohio State University. 2024. DACUM Research Chart Bank. Retrieved 2024-09-04 from https://cete.osu.edu/dacum-research-chart-bank/

[53] Shunichiro Tomura and Hoa Khanh Dam. 2024. Generating explanations for AI-powered delay prediction in software projects. In *Generative AI for Effective Software Development*, A Nguyen-Duc, P Abrahamsson, and F Khomh (Eds.). Springer Nature, Cham, Switzerland, 297–316. https://doi.org/10.1007/978-3-031-55642-5_14

[54] United Labor Agency and Education Development Center. 2017. *Profile of an employment specialist.* Education Development Center.

[55] Jules White, Sam Hays, Quchen Fu, Jesse Spencer-Smith, and Douglas C Schmidt. 2024. ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. In *Generative AI for Effective Software Development*, A Nguyen-Duc, P Abrahamsson, and F Khomh (Eds.). Springer Nature, Cham, Switzerland, 71–108. https://doi.org/10.1007/978-3-031-55642-5_4